Constructors and Destructors

Introduction.....2
Types of Constructors.....4
Default Constructor.....5
Parameterized Constructor.....6
Copy Constructor.....11
Shallow Copy.....13
Deep Copy.....13
Constructor Initialization List.....17
Constructor vs Member Initialization.....18
Example 1: Single Data Member.....21
Example 2: Multiple Members.....22
Example 3: Compare with Normal Assignment.....23
Method 1 (Assignment inside constructor).....23
Method 2 (Initialization List).....23
Example 4: Initializing Multiple Types.....24
Example 5: Required for const Members.....25
Example 6: Required for Reference Members.....26
Order of Initialization.....27
Advantages of Constructor Initialization List.....27
Constructor Delegation (C++11).....27
Rules of Constructor.....28
What is Destructor?.....28
Destructor for Dynamic Memory.....32
Constructor vs Destructor.....33
Order of Constructor and Destructor Calls in Inheritance.....33
Order of Constructor and Destructor Calls in Composition.....35
Practice Questions.....37

Introduction

When we create an object of a class, some initialization work is usually required. Similarly, when the object is destroyed, some cleanup work may also be needed.

C++ provides two special member functions for these purposes:

- **Constructor** → Initializes an object.d
- **Destructor** → Cleans up resources before an object is destroyed.

These functions are automatically called by the compiler.

What is a Constructor?

Definition

A **constructor** is a special member function of a class that is **automatically called whenever an object is created**.

Its primary purpose is to initialize the data members of the object.

Characteristics

- Same name as the class.
- No return type (not even `void`).
- Called automatically.
- Can be overloaded.
- Can have default arguments.
- Cannot be inherited.
- Cannot be virtual.
- Usually declared in the public section.

Syntax

```
class Student
{
public:

   Student()
   {
```

```
        // Constructor Body
    }
};
```

Why Constructors are Needed?

Without constructors:

```
class Student
{
public:
    int roll;
    float marks;
};

int main()
{
    Student s;

    cout << s.roll;
}
```

Output:

Garbage Value

Members are **uninitialized**.

Using a constructor:

```
class Student
{
public:

    int roll;
    float marks;

    Student()
    {
        roll = 1;
        marks = 90;
```

```
    }
};
```

Now every object automatically receives initialized values.

Constructor Execution

```
class Demo
{
public:

    Demo()
    {
        cout<<"Constructor Called";
    }
};

int main()
{
    Demo d;
}
```

Output

Constructor Called

Notice: We never called the constructor manually.

Types of Constructors

There are several types:

Constructor:

- Default Constructor
- Parameterized Constructor
- Copy Constructor
- Dynamic Constructor
- Constructor with Default Arguments

Default Constructor

Definition

A constructor with **no parameters** is called the default constructor.

Example

```
#include <iostream>
using namespace std;

// Define a class named Student
class Student
{
private:

    int roll;          // Stores the student's roll number
    float marks;       // Stores the student's marks

public:

    // Default constructor
    // It is automatically called when an object is created
    Student()
    {
        roll = 101;    // Initialize roll number
        marks = 95.5;  // Initialize marks
    }

    // Member function to display student details
    void display()
    {
        cout << "Roll : " << roll << endl;
        cout << "Marks : " << marks;
    }
};

int main()
{
    // Create an object of Student class
    // The default constructor is called automatically
    Student s;

    // Call the display() function using the object
    s.display();
```

```
    return 0;          // Indicate successful program execution
}
```

Output

Roll : 101
Marks : 95.5

Compiler Generated Default Constructor

If we do not write any constructor,

```
class Test
{
   int x;
};
```

Compiler automatically creates

```
Test()
{
}
```

This constructor does **nothing**. It does **not initialize** variables.

Parameterized Constructor

Definition

A constructor that accepts one or more parameters.

Example

```
#include <iostream>
using namespace std;

// Define a class named Student
```

```
class Student
{
private:

    int roll;          // Stores the student's roll number
    float marks;       // Stores the student's marks

public:

    // Parameterized constructor
    // It receives roll number and marks as arguments
    Student(int r, float m)
    {
        roll = r;      // Assign r to roll
        marks = m;     // Assign m to marks
    }

    // Member function to display student details
    void display()
    {
        cout << roll << " " << marks << endl;
    }
};

int main()
{
    // Create the first Student object
    // 101 is passed to r and 92.5 is passed to m
    Student s1(101, 92.5);

    // Create the second Student object
    // 102 is passed to r and 88.6 is passed to m
    Student s2(102, 88.6);

    // Display details of the first student
    s1.display();

    // Display details of the second student
    s2.display();

    return 0;          // End the program successfully
}
```

Output

101 92.5
102 88.6

Constructor Overloading

Like normal functions, constructors can also be overloaded.

Example

```
#include <iostream>
using namespace std;

// Define a class named Demo
class Demo
{
public:

    // Default constructor
    // It does not take any parameters
    Demo()
    {
        cout << "Default Constructor" << endl;
    }

    // Parameterized constructor
    // It takes one parameter
    Demo(int x)
    {
        cout << "Parameterized Constructor" << endl;
    }

    // Parameterized constructor
    // It takes two parameters
    Demo(int x, int y)
    {
        cout << "Two Parameter Constructor" << endl;
    }
```

```
};

int main()
{
    // Create an object using the default constructor
    Demo d1;

    // Create an object using the one-parameter constructor
    Demo d2(10);

    // Create an object using the two-parameter constructor
    Demo d3(10, 20);

    return 0;    // End the program successfully
}
```

Output

Default Constructor
Parameterized Constructor
Two Parameter Constructor

Constructor with Default Arguments

```
#include <iostream>
using namespace std;

// Define a class named Student
class Student
{
private:

    int roll;          // Stores the student's roll number

public:

    // Constructor with a default argument
    // If no value is passed, r will be 100
    Student(int r = 100)
    {
        roll = r;      // Assign the value of r to roll
```

```
    }

    // Member function to display the roll number
    void display()
    {
        cout << roll << endl;
    }
};

int main()
{
    // No argument is passed
    // Therefore, the default value 100 is used
    Student s1;

    // Argument 500 is passed
    // Therefore, r becomes 500
    Student s2(500);

    // Display the roll number of s1
    s1.display();

    // Display the roll number of s2
    s2.display();

    return 0;          // End the program successfully
}
```

Output
100
500

Copy Constructor

Definition

A constructor that creates one object from another object of the same class.

Syntax

```
ClassName(const ClassName &obj)
{
}
```

Example

```
#include <iostream>
using namespace std;

// Define a class named Student
class Student
{
private:

    int roll;          // Stores the student's roll number

public:

    // Parameterized constructor
    // It initializes the roll number
    Student(int r)
    {
        roll = r;      // Assign the value of r to roll
    }

    // Copy constructor
    // It creates a new object by copying data from another object
    Student(const Student &obj)
    {
        roll = obj.roll;   // Copy roll number from the existing object
    }

    // Member function to display the roll number
    void display()
    {
        cout << roll << endl;
    }
};

int main()
{
    // Create the first Student object
```

```
    // Parameterized constructor is called
    Student s1(101);

    // Create s2 as a copy of s1
    // Copy constructor is called
    Student s2 = s1;

    // Display the roll number of s1
    s1.display();

    // Display the roll number of s2
    s2.display();

    return 0;          // End the program successfully
}
```

Output

101
101

When Copy Constructor is Called?

It is called

- While initializing one object from another

Student s2=s1;

- Passing object by value

fun(Student s)

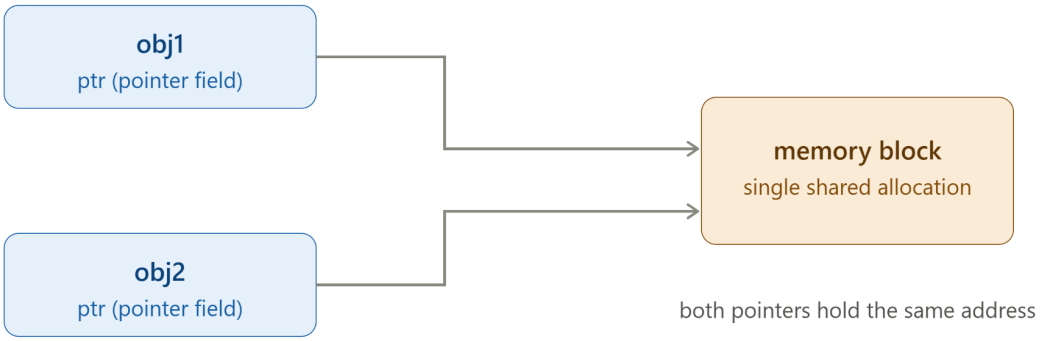
- Returning object by value

return s;

Shallow Copy

Compiler generated copy constructor performs **shallow copy**. It copies only values. If pointers are present, both objects point to the same memory.

Example

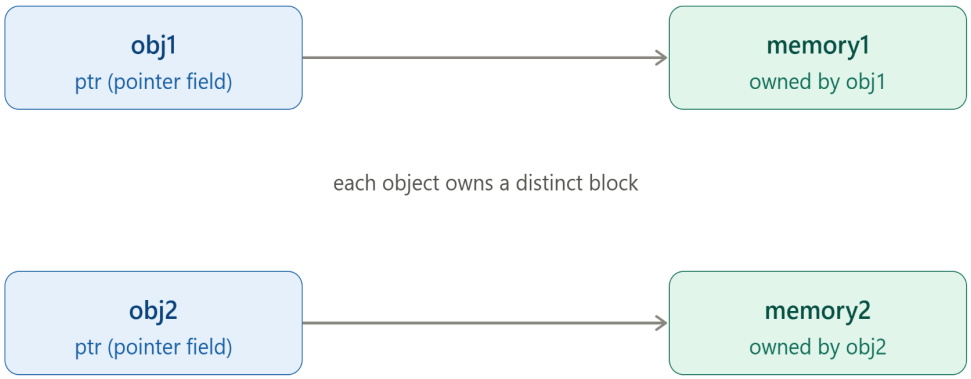


Problem:

Deleting memory using one object leaves another object with a dangling pointer.

Deep Copy

Deep copy creates separate memory.



Safe for dynamic memory allocation.

Example

```
#include <iostream>
using namespace std;

// Define a class named Demo
class Demo
{
```

```
private:

    int *ptr;          // Pointer to dynamically allocated memory

public:

    // Parameterized constructor
    Demo(int x)
    {
        // Dynamically allocate memory for one integer
        ptr = new int;

        // Store the value of x in the allocated memory
        *ptr = x;
    }

    // Copy constructor
    Demo(const Demo &obj)
    {
        // Allocate separate memory for the new object
        ptr = new int;

        // Copy the value stored in obj's memory
        // Only the value is copied, not the address
        *ptr = *obj.ptr;
    }

    // Member function to display the value
    void display()
    {
        cout << *ptr << endl;
    }

    // Destructor
    // It is automatically called when the object is destroyed
    ~Demo()
    {
        // Release the dynamically allocated memory
        delete ptr;
    }
};
```

```
int main()
{
    // Create the first object
    // Parameterized constructor is called
    Demo d1(50);

    // Create d2 as a copy of d1
    // Copy constructor is called
    Demo d2 = d1;

    // Display the value stored by d1
    d1.display();

    // Display the value stored by d2
    d2.display();

    return 0;
}
```

Dynamic Constructor

A constructor that allocates memory dynamically using new.

Example

```
#include <iostream>
using namespace std;

class Demo
{
private:

    int *arr;          // Pointer to dynamically allocated array

public:

    // Default constructor
    Demo()
    {
```

```
        // Allocate memory for 5 integers
        arr = new int[5];

        // Initialize the array
        for(int i = 0; i < 5; i++)
        {
            arr[i] = 0;
        }
    }

    // Function to display array elements
    void display()
    {
        cout << "Array elements: ";

        for(int i = 0; i < 5; i++)
        {
            cout << arr[i] << " ";
        }

        cout << endl;
    }

    // Destructor
    // It is automatically called when the object is destroyed
    ~Demo()
    {
        // Release the dynamically allocated array memory
        delete[] arr;

        cout << "Memory released" << endl;
    }
};

int main()
{
    // Create an object of Demo
    Demo d;

    // Display the array elements
    d.display();
```

```
    return 0;
}
```

Constructor Initialization List

Instead of assigning values inside constructor body,

use

```
ClassName(parameter_list) : member1(value1), member2(value2)
{
    // Constructor body
}
```

Example

```
#include <iostream>
using namespace std;

// Define a class named Student
class Student
{
private:

    int roll;          // Stores the student's roll number

public:

    // Parameterized constructor
    // The member initializer list initializes roll directly
    Student(int r) : roll(r)
    {
        // Constructor body
        // No assignment is required here because roll is already
initialized
    }
```

```
    // Member function to display the roll number
    void display()
    {
        cout << roll;
    }
};

int main()
{
    // Create a Student object
    // The constructor is called with 10 as the argument
    Student s(10);

    // Call the display() function using the object
    s.display();

    return 0;          // End the program successfully
}
```

Advantages

- Faster
- Required for const data members
- Required for reference members
- Preferred for member objects

Constructor vs Member Initialization

Less Efficient

```
Student(int r)
{
   roll=r;
}
```

More Efficient

```
Student(int r):roll(r)
{
}
```

Explicit Constructor

Used to prevent implicit conversion.

Without explicit

```
#include <iostream>
using namespace std;

// Define a class named Demo
class Demo
{
public:

    // Explicit constructor
    // It accepts one integer parameter
    explicit Demo(int x)
    {
        // Display the value passed to the constructor
        cout << x;
    }
};

int main()
{
    // Direct initialization
    // Explicit constructor can be called directly like this
    Demo d(10);

    return 0;          // End the program successfully
}
```

Compiler automatically converts 10 into object.

With explicit

```
#include <iostream>
using namespace std;

// Define a class named Demo
class Demo
{
public:

    // Parameterized constructor
    // It accepts one integer parameter
    Demo(int x)
    {
        // Display the value passed to the constructor
        cout << x;
    }
};

int main()
{
    // Create an object of Demo
    // The integer value 10 is passed to the constructor
    Demo d = 10;

    // Demo d = 10;
    // This is an example of implicit conversion
    // The compiler uses Demo(int x) to convert 10 into a Demo object

    return 0;          // End the program successfully
}
```

A **Constructor Initialization List** is a special syntax in C++ that initializes data members **before the constructor body executes**. It is the recommended way to initialize class members, especially for `const` members, reference members, and objects.

Syntax

```
ClassName(parameter_list) : member1(value1), member2(value2),
member3(value3)
{
   // Constructor body
```

```
}
```

General Form

```
class ClassName
{
private:
   data_type member1;
   data_type member2;

public:
   ClassName(parameters) : member1(value1), member2(value2)
   {
       // Optional code
   }
};
```

Example 1: Single Data Member

```
#include <iostream>
using namespace std;

class Student
{
private:
   int roll;

public:
   Student(int r) : roll(r)
   {
       cout << "Constructor Called" << endl;
   }

   void display()
   {
       cout << "Roll = " << roll << endl;
   }
};

int main()
```

```
{
   Student s1(101);
   s1.display();

   return 0;
}
```

Output
Constructor Called
Roll = 101

Here,

Student(int r) : roll(r)

means

roll = r;

but **roll is initialized directly**, not assigned later.

Example 2: Multiple Members

```
#include <iostream>
using namespace std;

class Student
{
private:
   int roll;
   float marks;

public:
   Student(int r, float m)
       : roll(r), marks(m)
   {
   }

   void display()
   {
       cout << "Roll  = " << roll << endl;
```

```
        cout << "Marks = " << marks << endl;
    }
};

int main()
{
    Student s1(10, 89.5);

    s1.display();

    return 0;
}
Output
Roll = 10
Marks = 89.5
```

Example 3: Compare with Normal Assignment

Method 1 (Assignment inside constructor)

```
class Student
{
    int roll;

public:
    Student(int r)
    {
        roll = r;
    }
};
```

Here,

1. `roll` is first initialized with a default value.
2. Then `roll = r` assigns a new value.

Method 2 (Initialization List)

```
class Student
{
   int roll;

public:
   Student(int r) : roll(r)
   {
   }
};
```

Here,

- `roll` is initialized directly with `r`.
- No extra assignment occurs.

This is more efficient.

Example 4: Initializing Multiple Types

```
#include <iostream>
using namespace std;

class Employee
{
private:
   int id;
   string name;
   double salary;

public:
   Employee(int i, string n, double s)
       : id(i), name(n), salary(s)
   {
   }

   void display()
   {
       cout << "ID     : " << id << endl;
       cout << "Name   : " << name << endl;
       cout << "Salary : " << salary << endl;
   }
```

```
};

int main()
{
   Employee e1(101, "Rahul", 45000);

   e1.display();
}
```

Example 5: Required for `const` Members

```
#include <iostream>
using namespace std;

class Test
{
private:
   const int x;

public:
   Test(int a) : x(a)
   {
   }

   void display()
   {
       cout << x;
   }
};

int main()
{
   Test t(50);

   t.display();
}
```

This works because `const` data members **must be initialized** in the initialization list.

The following **does not compile**:

```
Test(int a)
{
   x = a;      // Error
}
```

Example 6: Required for Reference Members

```
#include <iostream>
using namespace std;

class Demo
{
private:
   int &ref;

public:
   Demo(int &x) : ref(x)
   {
   }

   void display()
   {
       cout << ref;
   }
};

int main()
{
   int a = 25;

   Demo d(a);

   d.display();
}
```

Reference members must also be initialized using the constructor initialization list.

Order of Initialization

Members are initialized **in the order they are declared in the class**, **not** in the order written in the initialization list.

```
class Demo
{
   int a;
   int b;

public:
   Demo() : b(20), a(10)
   {
   }
};
```

Actual order of initialization:

1. `a` is initialized first.
2. `b` is initialized second.

because `a` is declared before `b`.

Advantages of Constructor Initialization List

- Initializes data members directly.
- More efficient than assigning values inside the constructor body.
- Required for `const` data members.
- Required for reference (`&`) data members.
- Required when initializing member objects that do not have a default constructor.
- Improves code readability and performance.

Constructor Delegation (C++11)

One constructor can call another constructor of the same class.

```
class Demo
{
public:

   Demo()
   {
       cout<<"Default";
   }

   Demo(int x):Demo()
   {
       cout<<x;
   }
};
```

Rules of Constructor

- Same name as class.
- No return type.
- Called automatically.
- Can be overloaded.
- Can have default arguments.
- Cannot be static.
- Cannot be virtual.
- Cannot be inherited.
- Usually public.

What is Destructor?

Definition

A destructor is a special member function automatically called when an object is destroyed.

Purpose:

- Free dynamically allocated memory
- Close files
- Release resources
- Perform cleanup

Syntax

```
class Demo
{
public:

   ~Demo()
   {

   }
};
```

Notice : ~ called **tilde**.

Characteristics of Destructor

- Same name as class
- Begins with ~
- No return type
- No parameters
- Only one destructor allowed
- Cannot be overloaded
- Automatically called
- Used for cleanup

Destructor Example

```
#include <iostream>
using namespace std;

// Define a class named Demo
class Demo
{
public:

    // Constructor
    // It is automatically called when an object is created
    Demo()
    {
        cout << "Constructor" << endl;
    }
```

```
    // Destructor
    // It is automatically called when the object is destroyed
    ~Demo()
    {
        cout << "Destructor" << endl;
    }
};

int main()
{
    // Create an object of Demo
    // Constructor is called automatically here
    Demo d;

    // When main() ends, object d is destroyed
    // Destructor is called automatically
    return 0;
}
```

Output
Constructor
Destructor

Destructor Execution Order

Objects are destroyed in **reverse order** of creation.

Example

```
#include <iostream>
using namespace std;

// Define a class named Demo
class Demo
{
private:

    int id;          // Stores the object ID

public:
```

```
    // Parameterized constructor
    // It is automatically called when an object is created
    Demo(int i)
    {
        id = i;      // Store the value of i in id

        // Display a message when the constructor is called
        cout << "Constructor " << id << endl;
    }

    // Destructor
    // It is automatically called when an object is destroyed
    ~Demo()
    {
        // Display a message when the destructor is called
        cout << "Destructor " << id << endl;
    }
};

int main()
{
    // Create the first object
    // Constructor is called for d1
    Demo d1(1);

    // Create the second object
    // Constructor is called for d2
    Demo d2(2);

    // Create the third object
    // Constructor is called for d3
    Demo d3(3);

    // When main() ends, objects are destroyed
    // Destructors are called in reverse order of creation

    return 0;
}
```

Output

Constructor 1
Constructor 2
Constructor 3

Destructor 3
Destructor 2
Destructor 1

Destructor for Dynamic Memory

Example:

```
#include <iostream>
using namespace std;

// Define a class named Demo
class Demo
{
private:

    int *ptr;          // Pointer to dynamically allocated integer memory

public:

    // Default constructor
    // It is automatically called when an object is created
    Demo()
    {
        // Dynamically allocate memory for one integer
        ptr = new int;
    }

    // Destructor
    // It is automatically called when the object is destroyed
    ~Demo()
    {
        // Release the dynamically allocated memory
        delete ptr;
    }
};
```

```
int main()
{
    // Create an object of Demo
    // Constructor is called automatically
    Demo d;

    // When main() ends, object d is destroyed
    // Destructor is called automatically
    return 0;
}
```

Constructor vs Destructor

Constructor	Destructor
Initializes object	Cleans object
Called during creation	Called during destruction
Same name as class	Same name prefixed with ~
Can have parameters	No parameters
Can be overloaded	Cannot be overloaded
Multiple constructors possible	Only one destructor
Used for initialization	Used for cleanup

Order of Constructor and Destructor Calls in Inheritance

Constructors execute from **base class to derived class**, while destructors execute in the **reverse order**.

```
#include<iostream>
```

```
using namespace std;

class Base
{
public:
   Base()
   {
       cout << "Base Constructor" << endl;
   }

   ~Base()
   {
       cout << "Base Destructor" << endl;
   }
};

class Derived : public Base
{
public:
   Derived()
   {
       cout << "Derived Constructor" << endl;
   }

   ~Derived()
   {
       cout << "Derived Destructor" << endl;
   }
};

int main()
{
   Derived d;
   return 0;
}
```

Output

Base Constructor
Derived Constructor
Derived Destructor
Base Destructor

Order of Constructor and Destructor Calls in Composition

When a class contains objects of other classes as members:

- Member objects are constructed **before** the enclosing class constructor body.
- Member objects are destroyed **after** the enclosing class destructor body, in reverse order of declaration.

```
#include<iostream>
using namespace std;

class Engine
{
public:
   Engine()
   {
       cout << "Engine Constructor" << endl;
   }

   ~Engine()
   {
       cout << "Engine Destructor" << endl;
   }
};

class Car
{
private:
   Engine e;

public:
   Car()
   {
       cout << "Car Constructor" << endl;
   }

   ~Car()
   {
       cout << "Car Destructor" << endl;
   }
```

```
};

int main()
{
   Car c;
}
```

Output

Engine Constructor
Car Constructor
Car Destructor
Engine Destructor

Constructor and Destructor Call Sequence

Object Created

Memory Allocated

Constructor Called

Object Used

Object Goes Out of Scope / delete

Destructor Called

Memory Released

Best Practices

- Always initialize data members using **member initializer lists** whenever appropriate.
- Release every resource acquired with `new` using the matching `delete` or `delete[]`.
- Prefer **deep copy** when a class owns dynamically allocated memory.

- Mark single-argument constructors as `explicit` to avoid unintended implicit conversions.
- Follow the **Rule of Three**: if you define a destructor, copy constructor, or copy assignment operator, you usually need all three.
- In modern C++, prefer using smart pointers (such as `std::unique_ptr`) instead of raw pointers whenever possible.

Practice Questions

1. What is a constructor?
2. Why are constructors needed?
3. What are the different types of constructors?
4. What is the difference between a default and a parameterized constructor?
5. What is a copy constructor?
6. What is shallow copy? Why is it dangerous?
7. What is deep copy?
8. What is a dynamic constructor?
9. What is a member initializer list? Why is it preferred?
10. What is an explicit constructor?
11. What is constructor delegation?
12. What is a destructor?
13. Why can't a destructor be overloaded?
14. In what order are constructors and destructors called in inheritance?
15. What is the Rule of Three? How does it relate to constructors and destructors?
16. What happens if a class with dynamically allocated memory does not define a destructor?